

Beyond E2E Scripts: Using LLM-Proposed Scenarios Without Letting the LLM Be the Oracle

Draft manuscript

May 2026

Abstract

Large language models can enlarge the scenario space explored by end-to-end tests, but they are unsafe as test oracles: they can invent impossible states, misread application semantics, and overstate weak evidence. Bayesilisk separates scenario proposal from verification. App-specific connectors publish a bounded search surface consisting of routes, actions, identifiers, state facts, invariants, and executable probe hooks. The Bayesilisk core expands that surface into candidate scenarios, rejects malformed candidates by deterministic contract validation, prioritizes valid probes with Grassmann-style attention and a Bayesian score, and verifies only evidence returned by connector execution. This paper gives a compact formal model for that loop and reports a Cal.com case study at a fixed May 2026 revision. From connector-declared route facts and an action graph, Bayesilisk generated seven probes: six unknown/stale `rescheduleUid` route mutations and one bounded workflow sequence that created a booking, cancelled it, and replayed the cancelled booking UID through a public booking route. All seven returned deterministic expected-vs-observed violations. One upstream report has a targeted fix pull request that identifies the root cause and adds a regression Playwright test. The result is not a claim that LLMs can judge application correctness; it is evidence that LLM-style proposal can be useful when the oracle remains deterministic and evidence-bound.

1 Introduction

End-to-end suites are reliable when they fail because their assertions are explicit, executable, and repeatable. Their weakness is coverage. A suite usually contains the workflows a team remembered to encode, not the combinatorial neighborhood around stale identifiers, role changes, cancelled state, direct routes, private links, feature flags, or replayed workflow tokens. LLM-driven browser agents have the opposite shape. They can suggest unusual states and longer sequences, but they are unreliable when asked to decide whether the observed behavior is correct.

Bayesilisk is built around a narrow separation: models may help propose where to look, but deterministic rules decide what counts as a failure. The connector for an application exposes context and executable hooks; it does not own the scenario search strategy. The Bayesilisk core generates candidate scenarios from the declared surface, validates candidates against the connector contract, ranks valid probes, asks the connector to execute them, and verifies returned evidence against explicit invariants.

This is close in spirit to property-based testing, where generators produce inputs and properties act as oracles [5]. The difference is the target: Bayesilisk treats a modern UI/API application as the system under test and uses browser/API probes as evidence sensors. It also treats LLM proposals as untrusted generator hints, not as properties and not as verdicts.

The practical value is that teams do not have to choose between narrow E2E scripts and unconstrained LLM agents. If a team can describe the app surface it already understands—actors, routes, actions, fixtures, identifiers, and rules—Bayesilisk can sweep the adjacent scenario space, prioritize suspicious combinations, and emit deterministic issue payloads. The claim is deliberately bounded but useful: Bayesilisk does not replace domain knowledge; it turns that domain knowledge into a larger, repeatable search procedure.

This paper makes four contributions:

1. a trust-boundary model for using LLM-proposed scenarios without using an LLM as the oracle;

2. a mathematical formulation of connector contracts, scenario validation, Grassmann-style attention, Bayesian ranking, and deterministic evidence verification;
3. a Cal.com artifact showing seven generated probes, seven observed expected-vs-observed violations, reproducible issue payloads, and an upstream targeted fix pull request for one reported issue; and
4. an explicit account of scope and generalization: connectors are app-specific by design, action-sequence breadth is governed by the connector-declared graph, and the Cal.com comparison runs validate the architectural separation from E2E-only, Playwright-only, weak-model, and LLM-agent workflows.

2 Claim Audit

A skeptical reader should ask four questions.

Who generates scenarios? The connector declares the app-specific search surface: routes, actions, identifiers, state facts, invariants, and valid mutation or action schemas. Bayesilisk generates concrete candidate scenarios from that declared surface, optionally using LLM proposals as additional candidate ideas. The connector does not enumerate final tests.

Who executes scenarios? The connector executes validated probe requests through the real application, local fixtures, Playwright, API calls, or a test harness. It returns observed evidence such as semantic status, URL, trace, screenshot, state transition, or execution error.

Who decides pass or fail? Bayesilisk decides pass/fail by applying deterministic invariants to evidence returned by the connector. Neither the LLM nor the connector is the oracle.

What external validation exists? The Cal.com artifact produced issue-ready findings. One public issue, #29399, has an upstream contributor fix pull request, #29400, that identifies a missing null guard in `processReschedule` and adds a regression Playwright test. A second workflow issue, #29407, remains open at capture time.

3 Formal Model

Let an application connector publish a contract

$$C = (R, A, F, I, M, G),$$

where R is a set of route patterns, A is a set of connector actions, F is a set of observable state facts and identifiers, I is a set of deterministic invariants, M is a set of parameter mutation schemas, and G is an action graph. The connector is app-specific; the Bayesilisk core is a generic function over C .

3.1 Scenario Generation and Validation

A route-mutation candidate has the form

$$s = (r, a, i, p, m),$$

where $r \in R$, $a \in A$, $i \in I$, p is a route parameter, and $m \in M(p)$ is an allowed mutation such as unknown, stale, foreign, missing, or duplicate. Bayesilisk generates candidates by taking the cross product of connector-declared facts and mutation schemas, then deduplicating by action, invariant, route pattern, parameter, and mutation id.

An action-sequence candidate is generated from an action graph

$$G = (A, \text{req}, \text{prod}),$$

where each action has required tokens and produced tokens. A sequence

$$\sigma = (a_1, \dots, a_k)$$

is valid for a terminal goal action a_k only if

$$\forall t \in \{1, \dots, k\} : \text{req}(a_t) \subseteq \bigcup_{j < t} \text{prod}(a_j), \quad k \leq d_{\max}.$$

Thus a connector can expose actions such as create booking, cancel booking, and open booking route, while Bayesilisk composes the bounded workflow. The connector still owns concrete execution.

Validation is deterministic:

$$V_C(s) = \begin{cases} 1 & \text{if every referenced route, action, parameter, fact, and invariant is declared in } C, \\ 0 & \text{otherwise.} \end{cases}$$

Only candidates with $V_C(s) = 1$ can become executable probes. LLM output enters the same validation gate. Unknown target planes, unknown ids, duplicate proposals, non-string fields, malformed JSON, impossible counts, and unsupported actions are rejected as proposer telemetry rather than reported as product failures.

3.2 Grassmann-Style Attention

The attention layer is a prioritizer, not an oracle. Conceptually, invariant descriptions and observations define local subspaces in a feature space. For an invariant plane P_i and context vector q , a Grassmannian proximity score can be written as

$$\gamma_i(q) = \|\Pi_{P_i} q\|_2^2,$$

where Π_{P_i} projects q onto the invariant plane. This follows the standard geometric view of subspaces on Grassmann manifolds [6].

The current implementation uses a bounded anchor-plane proxy, with optional local embedding similarity. For each invariant i , Bayesilisk computes

$$\alpha_i = \text{clip}_{[0,1]}(w_f f_i + w_u u_i + w_s \sigma_i + w_p p_i + w_n n_i - d_i),$$

where f_i is observed failure density, u_i is untestedness, σ_i is invariant sensitivity, p_i is Playwright evidence density, n_i is keyword or embedding novelty, and d_i is fixed/muted decay. The largest α_i values select planes for exploration. They do not determine whether a behavior is correct.

3.3 Bayesian Ranking

After a deterministic invariant result exists, Bayesilisk computes a ranking score. For invariant i , let π_i be its prior fault probability, let Δ_i be a bounded context adjustment, and let ℓ_i be the likelihood associated with the observed pass/fail result. The adjusted prior is

$$\tilde{\pi}_i = \text{clip}_{[0.05, 0.95]}(\pi_i + \Delta_i).$$

The posterior risk score is

$$\rho_i = \frac{\tilde{\pi}_i \ell_i}{\tilde{\pi}_i \ell_i + (1 - \tilde{\pi}_i)(1 - \ell_i)}.$$

This is a ranking score in the Bayesian sense [3]. It is not the pass/fail rule. Pass/fail is produced only by the invariant evaluator.

3.4 Evidence Verification

Execution is a connector function

$$X_C(s) \rightarrow e,$$

where e is observed evidence. For the Cal.com connector, the core evidence fields are `expectedStatus`, `observedStatus`, route, target URL, invariant id, and source context. The verifier applies

$$\phi_i(e) = \begin{cases} \text{fail} & \text{if } e.\text{observedStatus} \neq e.\text{expectedStatus}, \\ \text{pass} & \text{if } e.\text{observedStatus} = e.\text{expectedStatus}, \\ \text{invalid} & \text{if required evidence is absent or malformed.} \end{cases}$$

Only fail with evidence becomes an issue-ready finding. `invalid` is a connector/proposer diagnostic.

4 Implementation Boundary

Bayesilisk implements three distinct ledgers. The attention ledger records which invariant planes should be explored. The model-proposal ledger records accepted and rejected model proposals. The verification ledger records deterministic invariant outcomes, fingerprints, issue readiness, and risk scores. Only the verification ledger contains pass/fail results.

This boundary matters for connector design. A connector should expose a small vocabulary of actions and facts. It should not filter away uncomfortable cases or decide that a 200 response is acceptable. The finite-state verifier owns the ruleset, which is supplied by context and schema, not hardcoded for a particular application. App-specific code is isolated to connector execution.

5 Cal.com Case Study

We evaluated the current connector architecture on a local Cal.com checkout. The tested repository and revision were:

- Repository: <https://github.com/calcom/cal.com>
- Commit: 180ede28f0bddf2738933a6e60a8e80f6116d7da
- Commit date: 2026-05-14 19:30:21 +0000
- Commit subject: font-stack fallback fix for non-Latin script support (#29346)
- Bayesilisk seed: 150

The connector supplied three route facts for booking pages with `rescheduleUid` behavior and one action graph for a longer booking workflow. Bayesilisk generated six route-mutation probes and one workflow-sequence probe. The sequence was:

`create-booking → cancel-booking → open-public-booking-route.`

Artifact	Path
Connector code	<code>examples/calcom/bayesilisk-probes.e2e.ts</code>
Route context	<code>examples/calcom/source-context.json</code>
Generated route proposals	<code>examples/calcom/generated-proposals.json</code>
Sequence context	<code>examples/calcom/sequence-source-context.json</code>
Generated sequence proposal	<code>examples/calcom/generated-sequence-proposals.json</code>
Execution evidence	<code>examples/calcom/execution-context.json</code>
Bayesilisk report	<code>examples/calcom/reports/report.json</code>
Issue payloads	<code>examples/calcom/reports/issue-payloads.json</code>
Upstream outcomes	<code>examples/calcom/upstream-outcomes.md</code>

Table 1: Checked-in Cal.com artifact set.

All seven probes executed against local Playwright fixtures without unexpected Playwright test failures. All seven returned evidence that violated the connector-declared invariant. The route mutations expected semantic status 404 and observed 200. The cancelled-booking replay expected semantic status 409 and observed 200. Bayesilisk classified the findings as `breakage.context-observed` and produced issue-ready payloads.

6 Comparison Runs

The evaluation includes small comparison runs that clarify the role of Bayesilisk. These are not a broad benchmark; they are local controls around the Cal.com artifact.

Area	Mutation	Expected	Observed
Public booking page	unknown id	404	200
Public booking page	stale id	404	200
Private booking link	unknown id	404	200
Private booking link	stale id	404	200
Dynamic booking page	unknown id	404	200
Dynamic booking page	stale id	404	200
Cancelled booking replay	cancelled uid	409	200

Table 2: Verified Cal.com connector observations.

Workflow	Observation
Nearby E2E	14 nearby specs ran without unexpected failures; the existing suite did not cover the generated stale/unknown states.
Playwright-only probing	11 manual observations produced 4 failures; manual probes exposed only a subset of the bug family.
Bayesilisk route proposals	6 generated route proposals produced 6 findings across declared route variants.
Bayesilisk sequence proposal	1 generated workflow sequence produced 1 cancelled-booking replay finding.
Weak local model proposals	3 raw proposals produced 0 accepted probes because verifier validation rejected malformed candidates.
LLM-agent browser run	10 trials produced 3 oracle mismatches before deterministic verification.

Table 3: Cal.com comparison summary.

The comparison supports a bounded claim. Manual Playwright probes can find some instances when a human already knows where to look. Bayesilisk broadens coverage by expanding declared route facts and action graphs into concrete probes. The weak model result is also useful: invalid model proposals were contained by schema and verifier validation rather than converted into product bugs. The LLM-agent browser run illustrates why the model should not be the oracle.

7 Upstream Validation

Two upstream issues were opened from the Cal.com evidence. The first, #29399 [1], reported that an unknown `rescheduleUid` opened a normal booking flow instead of returning 404. An upstream contributor opened PR #29400 [4]. The PR identifies the root cause as a missing null guard in `processReschedule`, changes the server-side behavior to return `notFound`, and adds a Playwright regression test. The private-link variant was added to the same issue thread to avoid duplicate reporting. The second issue, #29407 [2], reports the cancelled booking UID replay sequence and remains open at capture time.

Finding group	Upstream response
Public unknown/stale <code>rescheduleUid</code>	Issue #29399 with targeted fix PR #29400: null guard and regression test
Private-link <code>rescheduleUid</code>	Added to issue #29399 as a related comment to avoid duplicate issue noise
Cancelled booking UID replay	Separate state-transition report in issue #29407; open at capture time

Table 4: Upstream response to Bayesilisk-generated findings.

This validation is practical rather than statistical. A fix PR does not prove the entire methodology, but it does show that at least one generated finding was actionable enough for an upstream contributor to diagnose

and patch.

8 Scope and Generalization

Bayesilisk is intended to be adopted through app-specific connectors and app-agnostic verification. The following points describe that engineering boundary and the evidence needed to broaden the empirical case; they are not exceptions to the architecture.

Application scope. The current public practical validation is Cal.com. The result does not depend on Cal.com-specific verifier code: the core consumes a connector contract and generated probes through the same app-agnostic interfaces used elsewhere in Bayesilisk. Additional application artifacts would strengthen empirical breadth, but the required adaptation is the expected one: each app team writes a connector that exposes its own routes, actions, state facts, fixtures, and invariants.

Connector specificity. Connector-specific code is not a weakness of the architecture; it is the integration boundary. A team that knows the app specification can declare actors, routes, action hooks, identifiers, and invariants without modifying the Bayesilisk core. Connector quality still matters: the broader and more accurate the declared surface, the broader the generated probe space.

Bounded workflow generation. Sequence generation is intentionally bounded by the connector-declared action graph and depth. That bound can still cover a large workflow space when the connector exposes a broad action vocabulary and meaningful state tokens. The design avoids arbitrary free-form browser wandering; Bayesilisk composes declared actions and the connector executes them.

Oracle scope. The deterministic oracle is only as good as the supplied ruleset. Bayesilisk prevents the LLM from being the oracle; it does not remove the need for correct invariants.

Baseline interpretation. The Cal.com comparison runs validate the architectural difference: nearby E2E did not cover the generated states, manual Playwright probes were narrower, weak local model output was rejected by validation, and browser-driving LLM trials showed oracle instability. A larger multi-app study would quantify effect size across projects and models; it is not required to establish the core separation demonstrated here.

9 Conclusion

Bayesilisk treats LLMs and coding agents as scenario proposers, not as test oracles. The mathematical boundary is simple: candidate generation and attention may be probabilistic or heuristic, but validation and verification are deterministic functions over a connector-declared contract and connector-returned evidence. The Cal.com case study demonstrates that this separation can produce useful real-app findings: seven generated probes, seven observed invariant violations, issue-ready payloads, and a targeted upstream fix PR for one reported issue. The adoption path is incremental: write the connector around existing app fixtures, let Bayesilisk expand the scenario space, and keep the oracle deterministic. The next research step is to repeat this artifact discipline across additional applications while keeping the Bayesilisk core app-agnostic.

References

- [1] Bayesilisk contributors. Cal.com issue 29399: Booking page opens instead of 404 for missing rescheduleuid. <https://github.com/calcom/cal.diy/issues/29399>, 2026. Issue report generated from Bayesilisk evidence.
- [2] Bayesilisk contributors. Cal.com issue 29407: Cancelled booking uid replay opens public booking route. <https://github.com/calcom/cal.diy/issues/29407>, 2026. Issue report generated from Bayesilisk workflow-sequence evidence.
- [3] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [4] Cal.com contributor. Cal.com pull request 29400: Return 404 for unknown rescheduleuid instead of opening booking flow. <https://github.com/calcom/cal.diy/pull/29400>, 2026. Human-authored fix pull request responding to Bayesilisk evidence.

- [5] Koen Claessen and John Hughes. Quickcheck: A lightweight tool for random testing of haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279. ACM, 2000.
- [6] Alan Edelman, Tomás A. Arias, and Steven T. Smith. The geometry of algorithms with orthogonality constraints. *SIAM Journal on Matrix Analysis and Applications*, 20(2):303–353, 1998.